

Architecture & API Reference

System Architecture

1. System Overview

AI Employee OS utilizes a modern, decoupled architecture.

- **Frontend:** Next.js (App Router) serving the presentation layer.
- **Backend:** FastAPI acting as the business logic, API, and AI orchestration layer.
- **Database:** Supabase (PostgreSQL) handling persistence, authentication, and security via RLS.
- **Workers:** Celery handling background asynchronous tasks (e.g., embeddings, email ingestion).

2. Layered Architecture

```
```mermaid.
```

```
graph TD.
```

```
Client[Web Browser] -->|HTTPS| Frontend[Next.js Frontend].
```

```
Frontend -->|JWT + HTTPS| Backend[FastAPI Backend].
```

```
Frontend -->|Auth / Direct Reads| Supabase[Supabase API]
```

```
Backend -->|SQLAlchemy| Postgres[(PostgreSQL)].
```

```
Backend -->|Tasks| Redis[Redis Broker].
```

```
Redis --> Workers[Celery Workers].
```

```
Workers -->|SQLAlchemy| Postgres.
```

```
Backend -->|API Calls| LLMs[OpenRouter / OpenAI / Anthropic]
```

```
```.
```

3. Request Lifecycle

1. **Authentication:** The user logs in via the Next.js frontend, which directly calls Supabase Auth.

Supabase returns a JWT.

2. **API Request:** The frontend makes a REST API request to the FastAPI backend, placing the Supabase JWT in the `Authorization: Bearer` header.

3. **Middleware Interception:** The FastAPI middleware intercepts the request.

The `get\_current\_user` dependency decodes the JWT using the `SUPABASE\_JWT\_SECRET` and extracts the `user\_id` and `organization\_id`.

4. **Business Logic:** The backend executes the requested logic (e.g., retrieving leads) using SQLAlchemy.

5. **Database Interaction:** SQLAlchemy queries the Postgres database using a service-role/superuser connection.

6. **Response:** The backend formats the data using Pydantic schemas and returns JSON to the frontend.

4. Database Architecture

- **Multi-Tenancy:** Achieved exclusively through Row Level Security (RLS). Every table (except platform-level tables) contains an `organization\_id` column.

- **RLS Policies:** Policies force `organization\_id = current\_org\_id()`. The `current\_org\_id()` is a Postgres SECURITY DEFINER function that reads the organization of the currently authenticated user (via `auth.uid()`).

- **Vector Storage:** The `knowledge_articles` and `ai_memories` tables utilize the `pgvector` extension to store text embeddings for Retrieval-Augmented Generation (RAG).

5. AI Architecture

```mermaid.

sequenceDiagram.

participant User.

participant API.

participant Orchestrator.

participant Agent.

participant Tools.

participant LLM.

User->>API: Send Chat Message.

API->>Orchestrator: Route Request.

Orchestrator->>Agent: Invoke specific agent (e.g., Sales)

Agent->>LLM: Generate plan/response with Tools

LLM-->>Agent: Request Tool Execution (e.g., search\_crm)

Agent->>Tools: Execute search\_crm()

Tools-->>Agent: Return CRM data.

Agent->>LLM: Provide Tool output.

LLM-->>Agent: Final textual response.

Agent-->>API: Yield streaming response.

API-->>User: Stream tokens to UI.

```.

6. Deployment Architecture (Target)

- **Frontend Hosting:** Vercel (Optimized for Next.js).

- **Backend Hosting:** Render, Railway, or AWS ECS running Docker containers (Web API and Celery Worker).

- **Database:** Supabase Cloud (Managed PostgreSQL).

- **Cache/Broker:** Redis (Upstash or Redis Labs).

API_SPEC.md

This document specifies the required REST API endpoints for the AI Employee OS backend.

These endpoints are strictly derived from the requirements of the frontend components and the existing database schema.

Core & Identity

`GET /api/v1/auth/me`

Purpose: Fetch the currently authenticated user's profile, roles, and organization context.

Authentication: Required (Supabase JWT).

Request: None.

****Response:**** `{ "id": "uuid", "email": "str", "organization\_id": "uuid", "roles": ["str"] }`.
****Validation:**** JWT signature and expiration against `SUPABASE\_JWT\_SECRET`.
****Errors:**** 401 Unauthorized (Invalid JWT).
****Database tables:**** `users`, `user\_roles`.
****Frontend Pages:**** Global (Session Provider, `/login`)
****Priority:**** Critical.

`POST /api/v1/organizations`

****Purpose:**** Create a new organization during the self-serve signup flow.
****Authentication:**** Required (Supabase JWT - user may not have an org yet).
****Request:**** `{ "name": "str", "industry": "str" }`.
****Response:**** `{ "id": "uuid", "name": "str", "created\_at": "timestamp" }`.
****Validation:**** `name` is required.
****Errors:**** 400 Bad Request.
****Database tables:**** `organizations`.
****Frontend Pages:**** `/register`.
****Priority:**** Critical.

AI Engine

`POST /api/v1/ai-chat/conversations`

****Purpose:**** Initialize a new chat session with a specific AI Agent.
****Authentication:**** Required.
****Request:**** `{ "agent\_id": "uuid", "title": "str" }`.
****Response:**** `{ "id": "uuid", "agent\_id": "uuid", "title": "str", "created\_at": "timestamp" }`.
****Validation:**** `agent\_id` must exist in `ai\_employees` for the current org.
****Errors:**** 404 Agent Not Found.
****Database tables:**** `ai\_conversations`, `ai\_employees`.
****Frontend Pages:**** `/dashboard/chat`.
****Priority:**** Critical.

`POST /api/v1/ai-chat/messages`

****Purpose:**** Send a message to an AI Agent and stream the response.
****Authentication:**** Required.
****Request:**** `{ "conversation\_id": "uuid", "content": "str" }`.
****Response:**** HTTP Chunked Transfer Encoding (Streaming text).
****Validation:**** User must own the `conversation\_id`.
****Errors:**** 403 Forbidden.
****Database tables:**** `ai\_messages`.
****Frontend Pages:**** `/dashboard/chat`.
****Priority:**** Critical.

CRM Domain

`GET /api/v1/crm/customers`

****Purpose:**** List customers for the current organization.

****Authentication:**** Required.

****Request:**** Query Params: `?limit=10&offset;=0`.

****Response:**** `{ "items": [{"id": "uuid", "name": "str", "email": "str", "status": "str"}], "total": int}`.

****Validation:**** None.

****Errors:**** None.

****Database tables:**** `customers`.

****Frontend Pages:**** `/dashboard/crm`.

****Priority:**** High.

`GET /api/v1/crm/leads`

****Purpose:**** List leads for the current organization.

****Authentication:**** Required.

****Request:**** Query Params: `?status=new`.

****Response:**** `{ "items": [{"id": "uuid", "name": "str", "score": int}], "total": int}`.

****Validation:**** None.

****Errors:**** None.

****Database tables:**** `leads`.

****Frontend Pages:**** `/dashboard/crm`.

****Priority:**** High.

Settings

`GET /api/v1/users`

****Purpose:**** List all members of the current organization.

****Authentication:**** Required (Org Admin role).

****Request:**** None.

****Response:**** `{ "items": [{"id": "uuid", "email": "str", "role": "str"}]}`.

****Validation:**** Caller must have admin privileges in the organization.

****Errors:**** 403 Forbidden.

****Database tables:**** `users`, `user\_roles`.

****Frontend Pages:**** `/dashboard/settings`.

****Priority:**** Medium.